Assignment #3: HTTP Proxy Server

Goal

In this assignment, you will build on your previous experiences by creating a HTTP proxy server. This server will receive HTTP requests from a client, pass on the request to the correct server, and then forward the response back to the client. To the real web server, your program will appear to be the client, and to the requester, your program will appear to be the web server.

Technology

You will implement your HTTP proxy using Visual Studio, C++, and Windows. Your submission will be validated on a DigiPen lab-machine-equivalent spec.

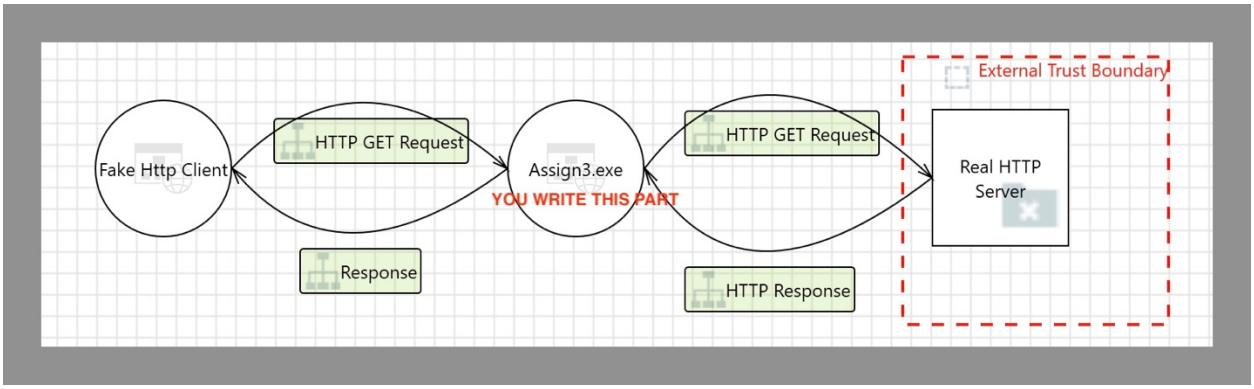
This assignment is more complex than the others, and it may make sense to create code organization (C++ classes, functions, etc.) around some of the concepts involved. However, it is a slippery slope to taking too much time to build a powerful general-purpose application. Stay focused on what you *need* to build for this assignment!

You *should not* directly copy/paste code from the textbook for this assignment. You may not use any AI assistance for this project.

I recommend using std::thread for creating threads, rather than fork or other techniques. Recall that you can pass parameters to the thread function. *Don't forget to detach() your thread after you create it!*

Tasks

Your program must handle simultaneous requests. You **cannot** block on accepting a new client, nor can clients block each other or the accept loop. *Note that this includes DNS lookups.*



Your server program will send the HTTP response it receives to the requester, but it should not output it to console. Your program's *only* normal console output should be "**Listening on port *n*",** where *n* is the port provided on the command line.

All HTTP requests you receive will contain a "Connection: close" header. Therefore, the webservers you are connecting to will automatically close the socket when it has sent the response. The overall HTTP request will look *something like*:

```
GET resource HTTP/1.1\r\n
Host: hostname\r\n
Connection: close\r\n\r\n
```

Your server program should run forever until stopped. You can do this by an infinite loop (i.e., *while (true)*), which can be broken with Ctrl-C without any work on your part. You do not need to worry about cleaning up any sockets or state when Ctrl-C is pressed.

Task Breakdown

Your program will need to go through these steps:

1. Starting the Server

Parse the argument (argv[1]) to determine which port you should bind to. If the argument is invalid, your program should output an explanation of how to run your program correctly and exit 1. Important: remember that the minimum value is 1, and the maximum value is 65535. If your program continues given an invalid port, points will be deducted. Invalid inputs like “abc”, “80abc”, “0”, “65536” should result in an error message and exit.

2. Begin Listening for Connections

Construct a listening address, using **INADDR_LOOPBACK** as the SOCKADDR_IN value, and the port provided. *Set the listen socket as non-blocking*. Your server should not be listening on any other addresses besides loopback.

3. Accept incoming connections

In the loop, accept() a new client socket in a non-blocking fashion. When a client socket arrives, handle it as defined below (next step). Note: The work for a client *must not* block the server from accepting new clients!

If accept() results in WSAEWOULDBLOCK, sleep 100 milliseconds before continuing.

4. Handle Client Requests

For each client: read the entire HTTP request from the client socket. The requests are GET only with no body, so the request is completewhen you’ve seen \r\n\r\n. Then parse the HTTP request to find the Host header.

4a. Request IP or Hostname

Resolve the address for the hostname: you should try inet_pton first, and if that fails, fall back to getaddrinfo. It could be an IP address or a DNS name.

4b. Connect to External Web Server

Construct a new socket and connect it to the correct address on port 80. Send *the entire HTTP request you received* to the correct server, as-is. Loop until the entire request is sent.

As the response arrives, begin sending it to the client. You do not need to retain the data you receive from the server. In fact, you should NOT do this – imagine if the request was to download a file of many terabytes!

The response is complete when the server socket *recv()* returns 0.

4c. Closing Connections Gracefully

Because every request includes a header for *Connection: close*, the web server will close its side when the response is done. You'll see that as *recv()* returning 0 on the server socket. At that point:

- Make sure every byte of the response has been *send()*'d to the client
- Call *shutdown(clientSocket, SD_SEND)* so the client receives a FIN and knows the response is complete
- Call *closesocket()* on both sockets.

You may also call *shutdown(serverSocket, SD_SEND)* after forwarding the request, but it isn't required. Do not use *SD_RECEIVE* or *SD_BOTH*. On Windows, if data arrives after the receive side is shut down, the connection is reset instead of closed gracefully. Also, calling *closesocket()* while unread data is still waiting on a socket causes a reset (RST) instead of a normal close.

4d. Threading Approaches

The responses may take an arbitrary amount of time. Therefore, it is *very* important that each client's work does not block other clients or the infinite-listening-loop, nor does the listening loop block the clients.

One Thread per Client

Once your program has handed off control to a new thread, that thread won't be blocking the main *accept()* loop. Accepted sockets inherit the non-blocking mode from the listen socket, so from that point forward, you can set the client socket back to being blocking, making the error handling code easier to work with.

4d. Other Notes

Note that your server does not care about the actual content of the HTTP response. HTTP error responses - 404-not-found, etc.... are all treated the same. Networking error scenarios will generally not be tested. As a best practice:

- Error handling in the construction of the server (before the infinite loop) should output an explanation and error code to the console and exit with status code 1.
- Error handling in the main thread should output an explanation and error code to the console and then continue.

Error handling during each client's request should output an explanation and error code to the console, with some context about the client, and then end the client request safely.

Testing

Test automation has been provided for you. You will need to **download the latest version of the FakeHttpClient.NET project** from:

```
https://github.com/DigiPen-Network-Classes/FakeHttpClient.net/releases
```

The FakeHttpClient sends HTTP 1.1 requests to your proxy server simultaneously and keeps track of the time for each. You don't need to compile this from source; the release version should be sufficient.

Things to Watch For

The blocking antipattern:

- Send request 1; your server starts request 1; this is maybe 5 seconds ...
- Meanwhile, Send request 2; your server is BUSY so it waits for 5 seconds before processing, total time = 10 seconds!
- Send request 3; same problem +5 seconds; etc.

Instead, the behavior we're looking for is: three (or more) requests are sent, each are processed at the same time, for a total elapsed time of around 5 seconds.

See the README at https://github.com/DigiPen-Network-Classes/FakeHttpClient.net for more information on how to run the tests.

Results

Per the automation, your program will be tested with a variety of requests, such as:

- http://www.google.com
- http://www.google.com/
- http://cs260.meancat.com/delay
- http://164.90.244.224/delay
- http://cs260.meancat.com/page-not-found

Other testing URLs may be used in assessment!

Note that in this assignment, only valid URLs will be tested, and all URLs will use http (not https).

As usual, your project must work in all configurations and architectures you provide. Note that the test automation is not a replacement for validating your build on all supported platforms for errors and warnings on all configurations!

Submission

Your project should be named **CS260_Assignment3**, and it should build **CS260_Assignment3.exe**. You should not do anything fancy with paths or copying the binary to other locations.

The files inside the submission zip should include:

- Your source code (likely just CS260_Assignment3.cpp)
- CS260_Assignment3.sln (or slnx)
- CS260_Assignment3.vcxproj
- CS260_Assignment3.vcxproj.filters
- CS260_Assignment3.vcxproj.user
- (optional) a README file

Note: there's no winsock documentation required.

Your submission should be just a few KBs, zipped. If your submission is larger, you likely included the hidden .vs directory or other build output.

You must name the archive **YourAlias-CS260-3.zip**, where YourAlias is your Moodle ID.

Grading

From a base score of 100, you'll lose points for the following errors:

- -100 if your program cannot be compiled in the Release configuration on *any* platform supported by your solution.
- -70 if your program doesn't open a listening TCP socket on the port specified by argv[1].
- -30 if your program allows invalid arguments for the port, without printing an error and terminating. A missing argument, non-numeric input, '80abc', 0, 65536, should all print an error and terminate.
- -10 if your program doesn't output "Listening on port *n*" to the console, where *n* is the specified port.
- -10 if your program outputs anything other "Listening on port *n*" to the console, such as debug information, during normal program operation. (Printing error messages is fine.)
- -70 if your program doesn't parse the incoming HTTP request and connect to the intended target server.
- Up to -40 if your program does not correctly and completely pass along the HTTP request to the web server, depending on how close you came.
- Up to -40 if your program does not correctly and completely pass along the HTTP response to the client, depending on how close you came.
- Up to -30 if your program closes either the server socket (proxy to web server) or the client socket (client to proxy) abnormally, depending on the nature of the problem.
- Up to -70 if your program does not process separate requests simultaneously, without any of the requests blocking the others, depending on how close you come.

Code Quality Penalties

In addition to the grading rubric above, you can receive these penalties based on code issues:

- -5 if your code has any large blocks of commented-out code. Your assignment should look "Finished" not half-done.
- -10 if your project does not compile cleanly, with no warnings at the default warning level 3
    - Do not change the default warning level for any assignment unless specified.
    - Warnings include any items in Visual Studio's Error List dialog, including Code Analysis as well as build output

Submission Penalties

In addition to the grading rubric above, you can receive these penalties if your submission is not correct:

- -10 if your submission file is not named as above.
- -10 if your executable is not named CS260_Assignment3.exe.
- -15 if your submission includes additional files.
- -15 if the test automation cannot build your project.